

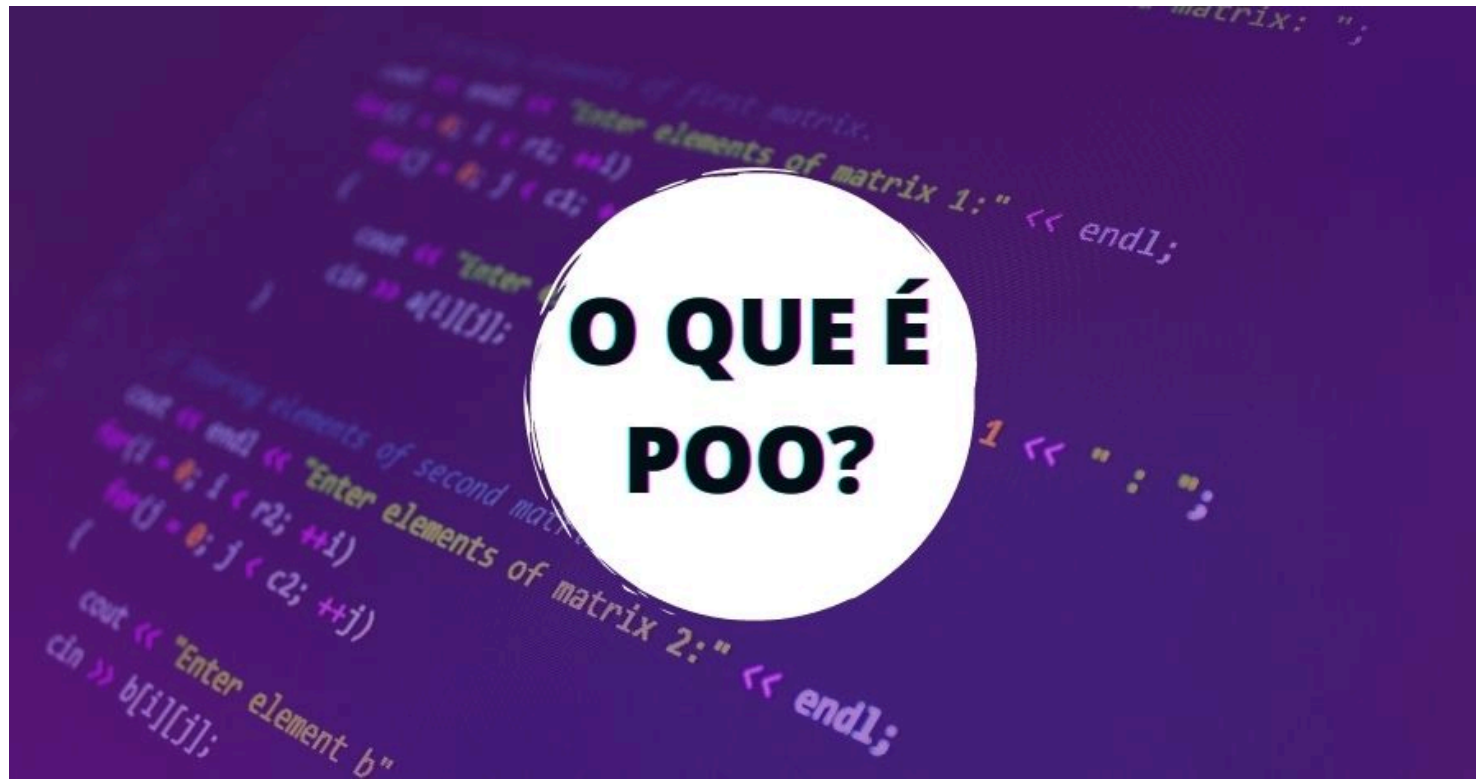


UNINASSAU

Revisão AV1

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

O QUE É PROGRAMAÇÃO ORIENTADA A OBJETOS (POO)?



- POO é um paradigma de programação que estrutura o código em torno de objetos, que representam entidades do mundo real ou conceitos abstratos, com características (atributos) e comportamentos (métodos). É uma forma de organizar o código para simplificar programas complexos, dividindo-os em unidades menores e reutilizáveis.
- Paradigma de programação baseado em **objetos**.
- Objetos possuem **atributos (dados)** e **métodos (ações)**.
- Facilita a modelagem de problemas reais.
- Utilizada em diversas linguagens modernas (Java, Python, C++, C#, etc.).

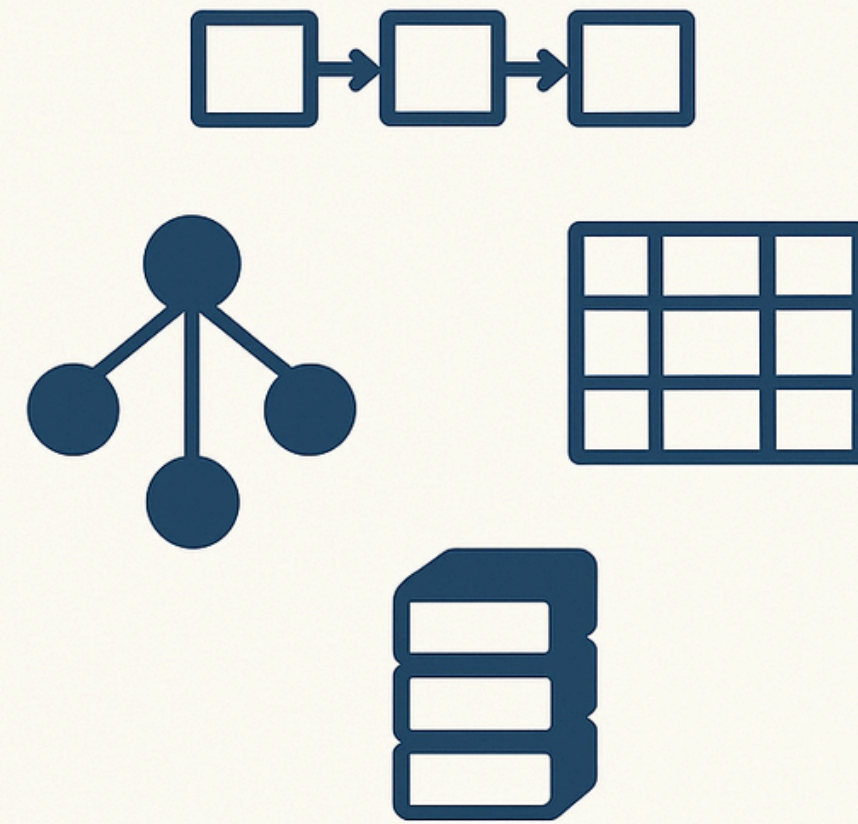
POO X ESTRUTURA DE DADOS

POO



Como organizar e modelar
o código em torno de
objetos

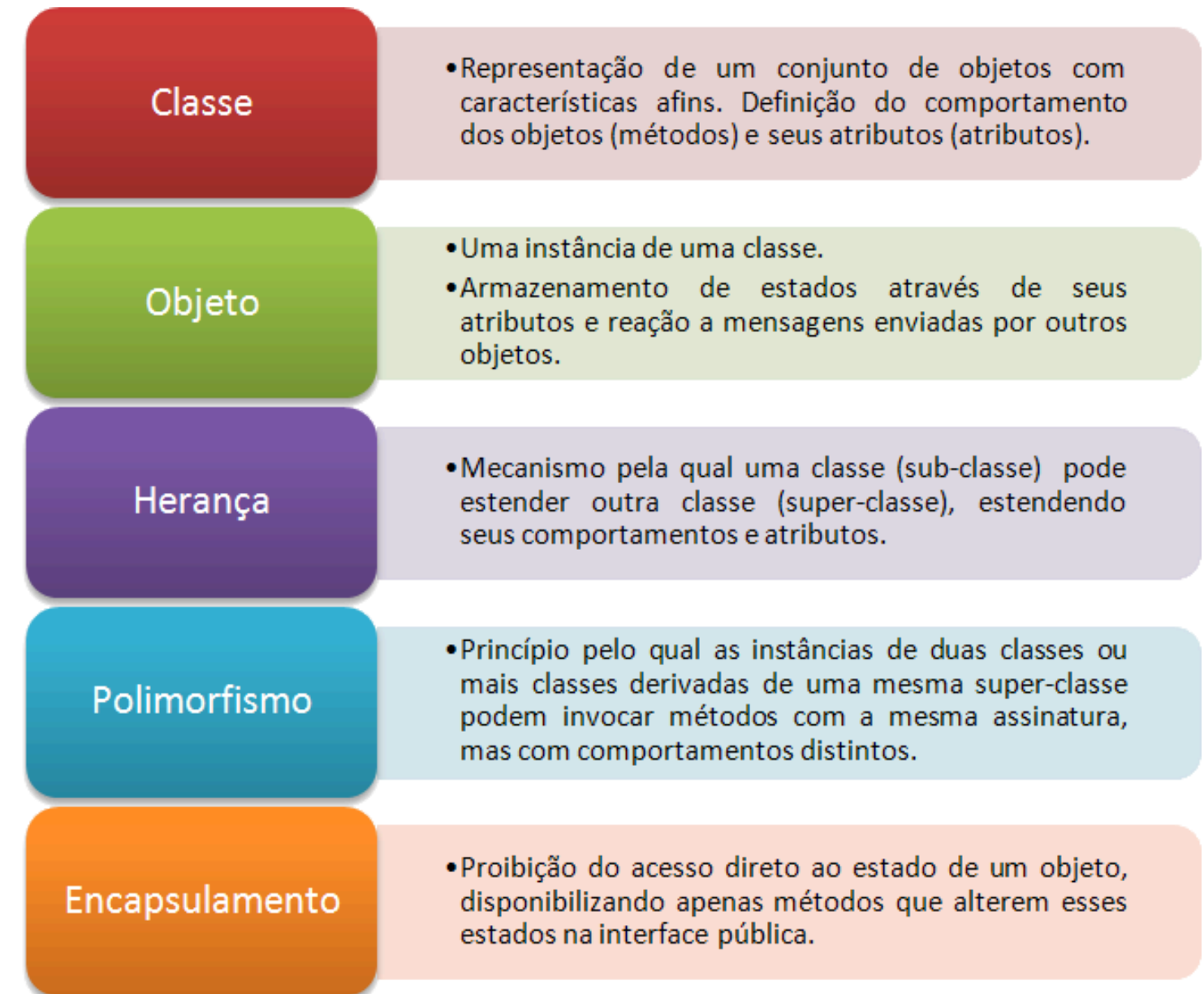
ESTRUTURA DE DADOS



Como armazenar e
organizar dados na memória

PRINCIPAIS CONCEITOS DA POO

- **Classe:** modelo/estrutura.
- **Objeto:** instância da classe.
- **Atributos:** características.
- **Métodos:** comportamentos.
- **Construtores:** inicialização de objetos.
- **Sobrecarga:** diferentes versões de métodos/construtores.



POO: CONCEITOS FUNDAMENTAIS

- **Classe:** Um molde ou projeto para criar objetos, definindo atributos e métodos.
- **Objeto:** Uma instância de uma classe, representando uma entidade do mundo real.
- **Atributos:** Variáveis que armazenam as características de um objeto.
- **Métodos:** Funções que definem o comportamento de um objeto.



OS QUATRO PILARES DE POO

- **Encapsulamento:** Oculta os detalhes internos de um objeto e protege os dados, garantindo a integridade do objeto através de interfaces públicas.
- **Herança:** Permite que uma classe (subclasse) herde atributos e métodos de outra classe (superclasse), promovendo a reutilização de código.



OS QUATRO PILARES DE POO

- **Polimorfismo:** Habilidade de objetos de diferentes classes responderem ao mesmo método de maneiras distintas, permitindo flexibilidade no código.
- **Abstração:** Foco nos aspectos essenciais de um objeto, ignorando detalhes irrelevantes, através de classes abstratas e interfaces.



EXEMPLO PRÁTICO: CARRO



Classe: Carro

Atributos: cor, modelo, ano

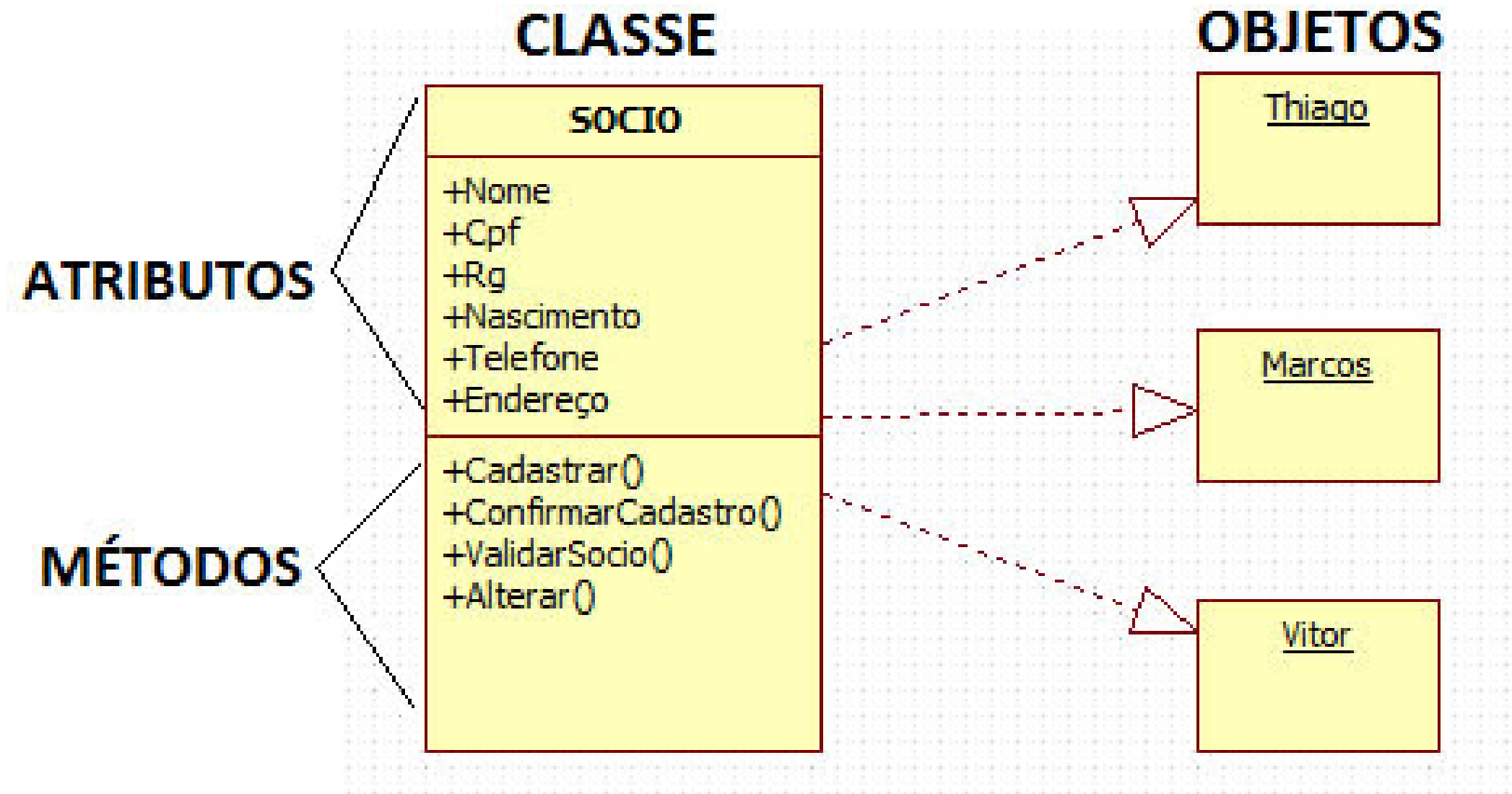
Método:
ligar()

```
class Carro {  
    String cor;  
    String modelo;  
    int ano;  
  
    void ligar() {  
        System.out.println("Carro ligado!");  
    }  
}
```

Classe

O QUE É UMA CLASSE?

- Uma classe é um modelo ou molde para criar objetos;
- Define atributos (dados) e métodos (ações);
- É como uma “fábrica” que produz objetos.



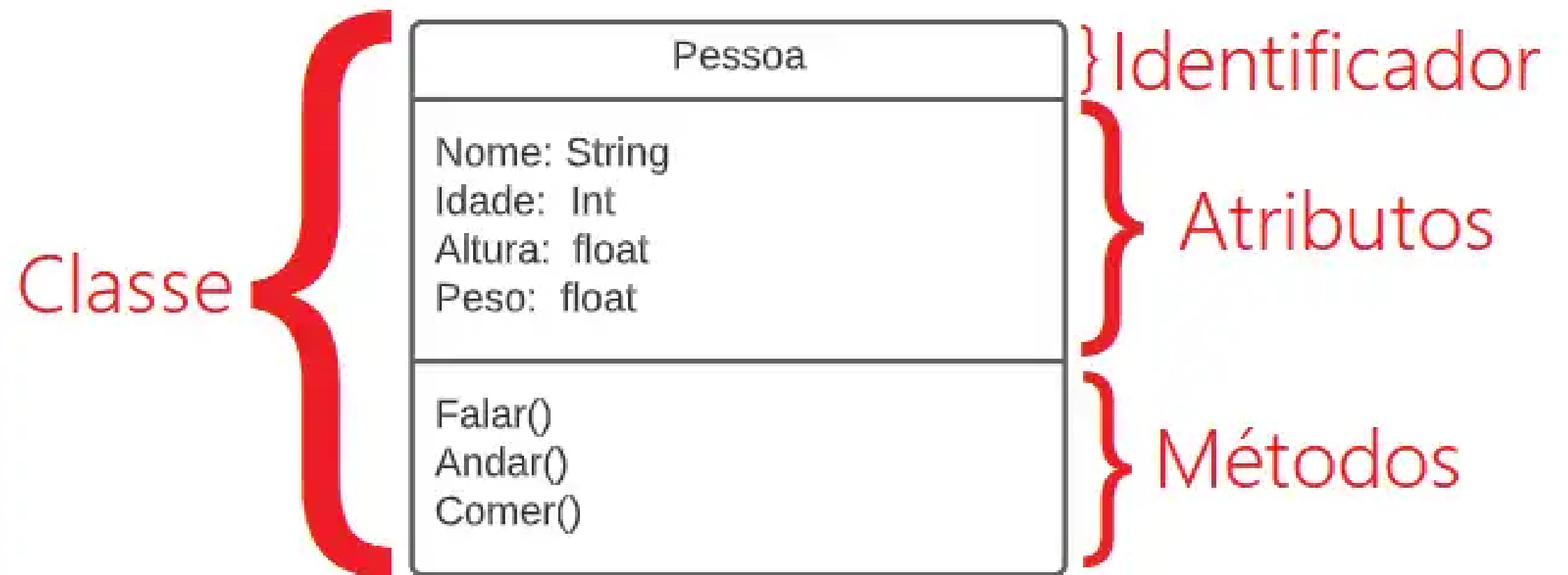
ESTRUTURA BÁSICA DE UMA CLASSE

- **Classe:** Aluno
- **Atributos:** nome, idade, curso
- **Métodos:** estudar(), assistirAula()

```
class Aluno {  
    String nome;  
    int idade;  
    String curso;  
  
    void estudar() {  
        System.out.println(nome + " está estudando.");  
    }  
  
    void assistirAula() {  
        System.out.println(nome + " está assistindo à aula.");  
    }  
}
```

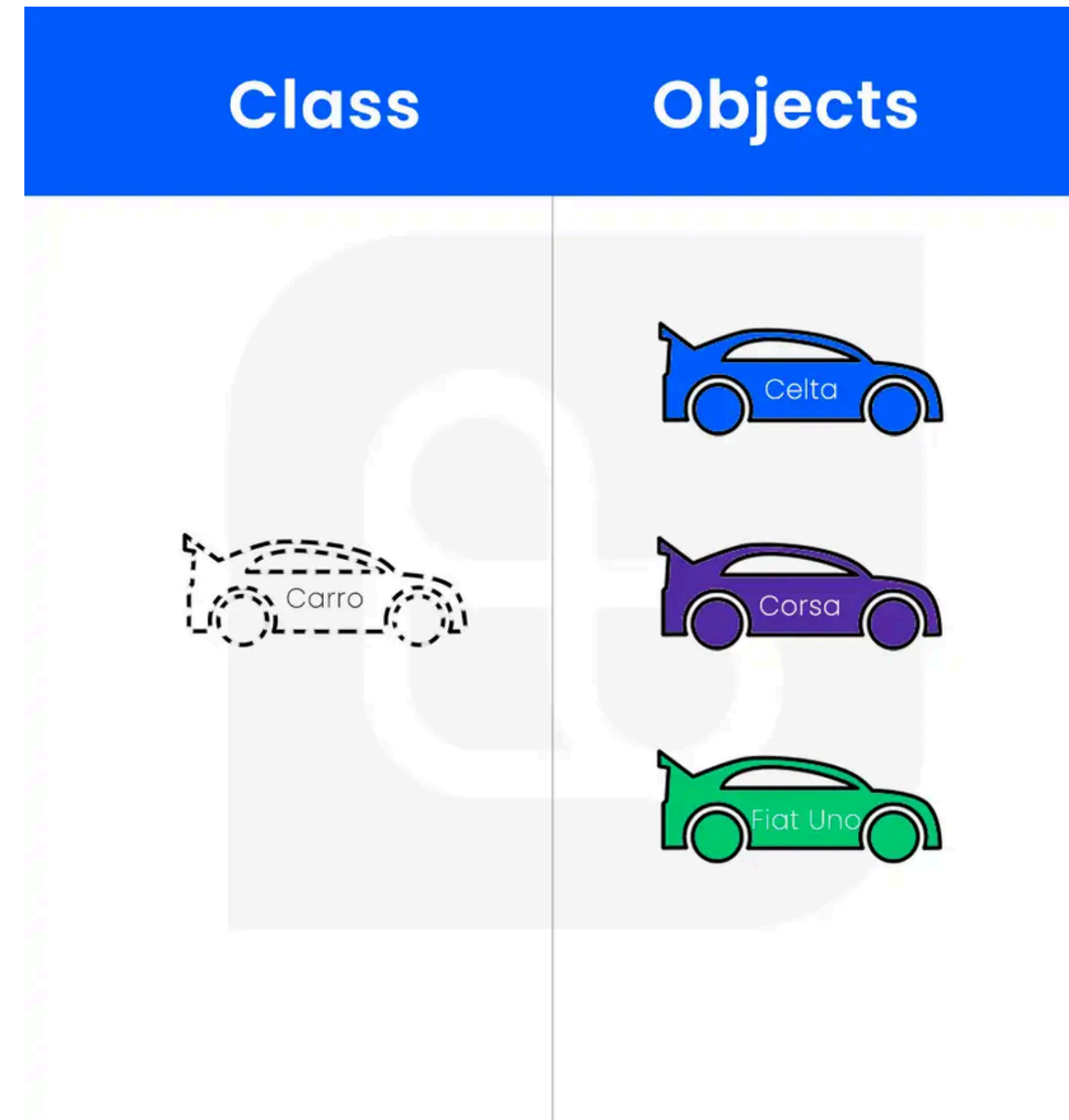
PAPEL DA CLASSE NA POO

- **Organizar** código.
- **Agrupar** dados e comportamentos.
- **Facilitar** a manutenção.
- **Reutilizar** em diferentes sistemas.



DIFERENÇA ENTRE CLASSE E OBJETO

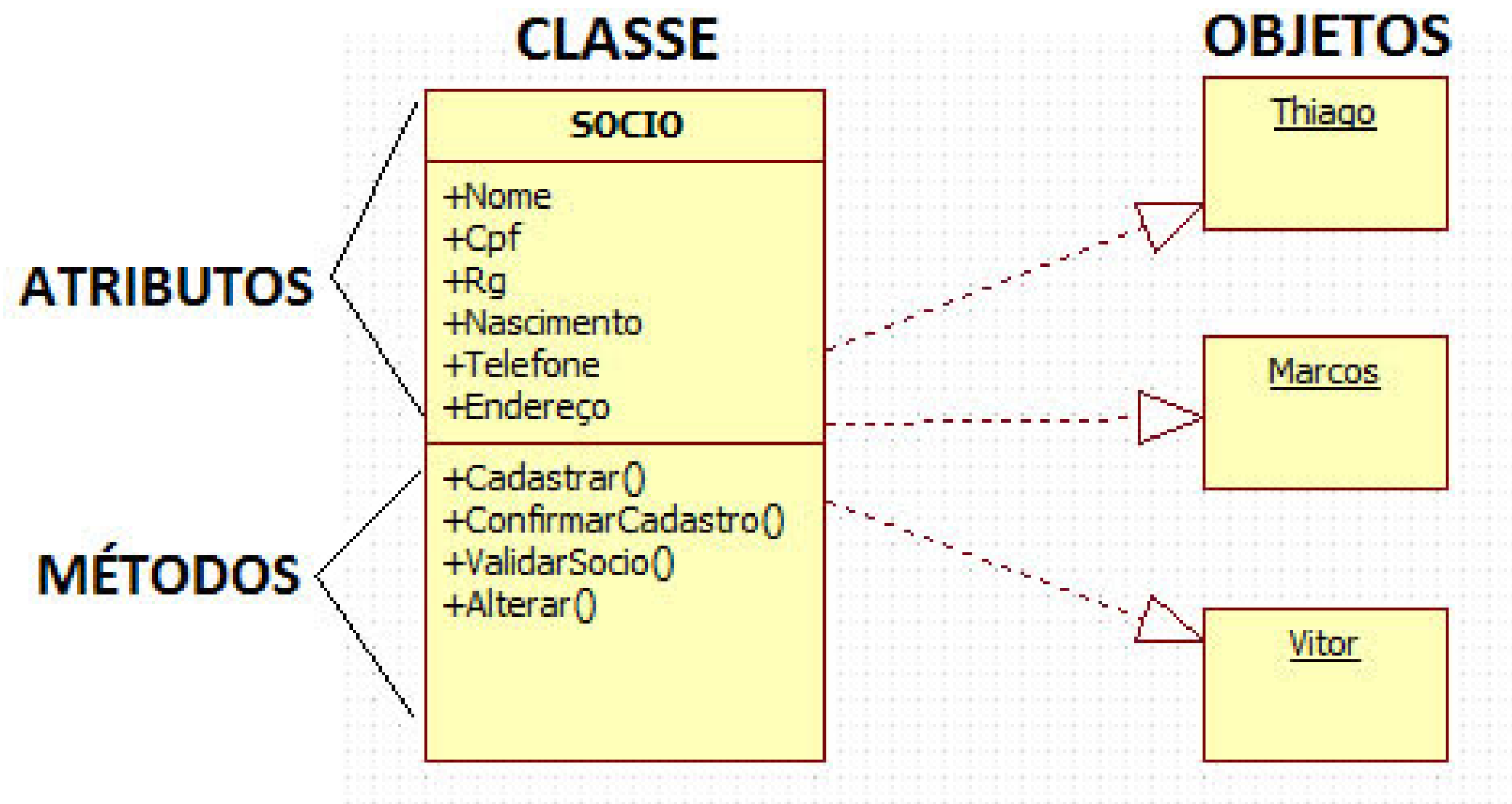
- **Classe:** modelo, estrutura, definição.
- **Objeto:** instância da classe (criado a partir dela).
- **Exemplo:**
 - Classe: Carro
 - Objetos: carro1, carro2, carro3



Objetos

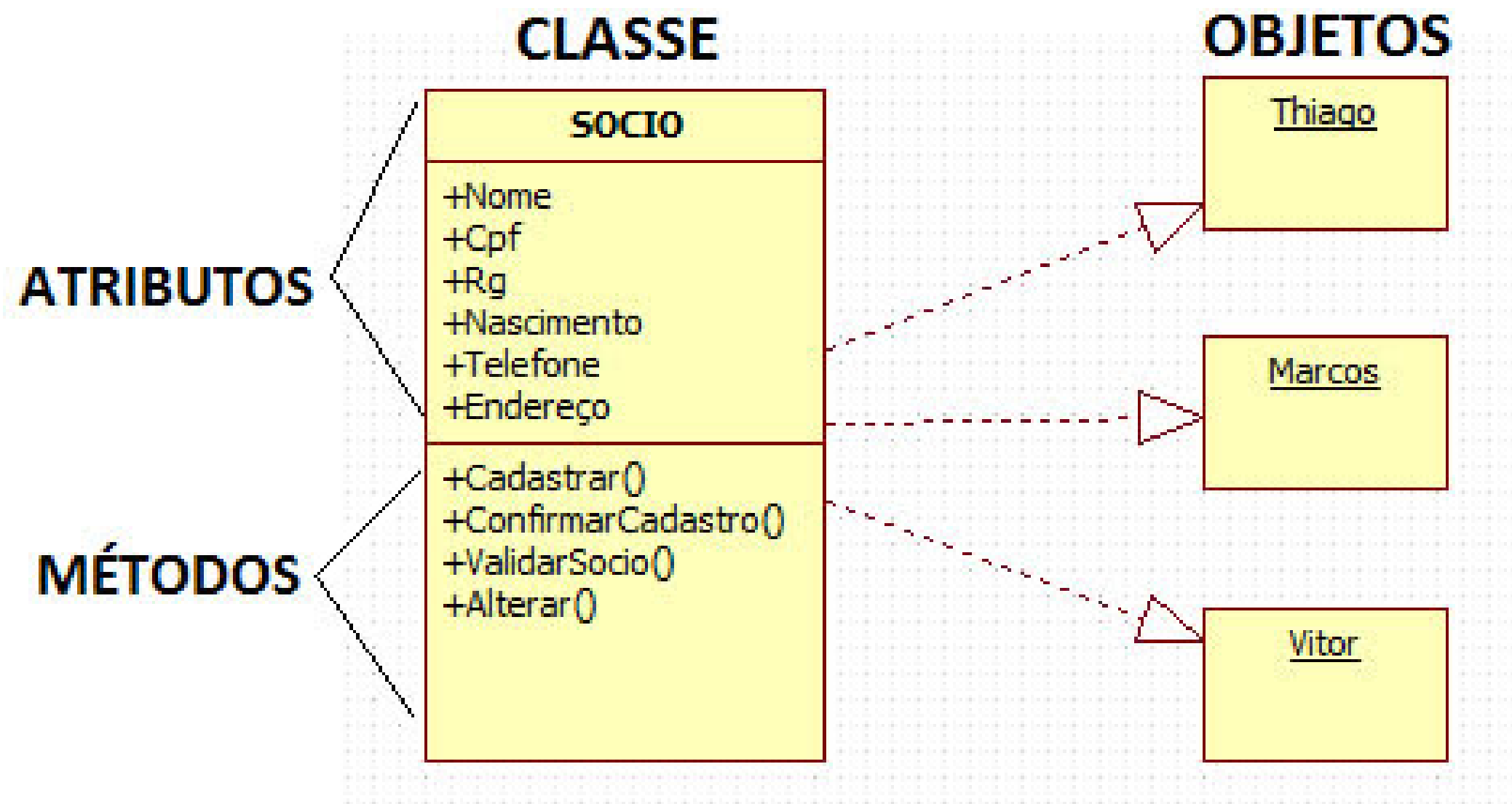
O QUE É UM OBJETO?

- Um **objeto** é uma **instância de uma classe**.
- Representa algo **concreto** no programa.
- Possui:
- **Atributos** → descrevem características.
- **Métodos** → definem comportamentos.



RELAÇÃO ENTRE CLASSE E OBJETO

- **Classe** = modelo (molde).
- **Objeto** = produto final criado a partir do molde.
- Analogia:
- Classe: planta de uma casa.
- Objeto: uma casa construída seguindo a planta.



ESTADO E COMPORTAMENTO DO OBJETO

- **Estado:** valores atuais dos atributos.
- Exemplo: nome = "Maria", curso = "Engenharia".
- **Comportamento:** ações realizadas pelos métodos.
- Exemplo: estudar(), assistirAula().



ESTADO E COMPORTAMENTO DO OBJETO

- **Estado:** valores atuais dos atributos.
- Exemplo: nome = "Maria", curso = "Engenharia".
- **Comportamento:** ações realizadas pelos métodos.
- Exemplo: estudar(), assistirAula().

Criação de Objetos em Código (Java)

```
Aluno aluno1 = new Aluno();  
aluno1.nome = "Maria";  
aluno1.idade = 20;  
aluno1.curso = "Engenharia";
```

VÁRIOS OBJETOS DA MESMA CLASSE

```
Aluno aluno1 = new Aluno();  
Aluno aluno2 = new Aluno();  
  
aluno1.nome = "Carlos";  
aluno2.nome = "Ana";
```

aluno1 e **aluno2** são diferentes objetos, mas da mesma classe Aluno.

Construtores e Sobrecarga

CONSTRUTORES

- Construtores são **métodos especiais** de uma classe.
- Usados para **inicializar objetos** no momento de sua criação.
- O nome do construtor é **igual ao nome da classe**.
- Não possui tipo de retorno (nem void).
- Pode receber **parâmetros** para inicializar atributos.
- É chamado automaticamente quando o objeto é criado.

```
class Pessoa {  
    String nome;  
    int idade;  
  
    Pessoa(String n, int i) {  
        nome = n;  
        idade = i;  
    }  
}
```

```
Pessoa p1 = new Pessoa("Ana", 25);
```

Criação de um objeto já inicializado.

CONSTRUTORES

Construtor Padrão

- Se nenhum construtor for definido, a linguagem cria um **construtor vazio**.

```
class Pessoa {  
    String nome;  
    int idade;  
}
```

```
Pessoa p = new Pessoa();
```

Construtor Personalizado

- Permite definir **valores iniciais**.

```
class Conta {  
    int numero;  
    double saldo;  
  
    Conta(int n, double s) {  
        numero = n;  
        saldo = s;  
    }  
}
```

EXEMPLO PRÁTICO DE CONSTRUTOR

```
class Conta {  
    int numero;  
    double saldo;  
  
    // Construtor  
    Conta(int n, double s) {  
        numero = n;  
        saldo = s;  
    }  
  
    // Método para exibir informações  
    void exibirDados() {  
        System.out.println("Conta: " + numero +  
                           " | Saldo: " + saldo);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // Criando objetos usando o construtor  
        Conta c1 = new Conta(1234, 500.0);  
        Conta c2 = new Conta(5678, 1500.0);  
  
        // Chamando método para mostrar os dados  
        c1.exibirDados();  
        c2.exibirDados();  
    }  
}
```

Saída Esperada:

```
Conta: 1234 | Saldo: 500.0  
Conta: 5678 | Saldo: 1500.0
```

SOBRECARGA DE CONSTRUTORES

- Uma classe pode ter **vários construtores**.
- Diferenciam-se pela **quantidade** ou **tipo dos parâmetros**.
- Benefícios:
 1. Mais flexibilidade na criação de objetos.
 2. Possibilita inicializações diferentes.
 3. Facilita a reutilização de código.

Uso da Sobrecarga

```
Aluno a1 = new Aluno();  
Aluno a2 = new Aluno("João", 20);
```

- **a1** usa o construtor sem parâmetros.
- **a2** usa o construtor com parâmetros.

Exemplo de Sobrecarga:

```
class Aluno {  
    String nome;  
    int idade;  
  
    Aluno() {  
        nome = "Sem nome";  
        idade = 0;  
    }  
  
    Aluno(String n, int i) {  
        nome = n;  
        idade = i;  
    }  
}
```

SOBRECARGA DE MÉTODOS

- Um método pode ter várias versões.
- Diferença ocorre na lista de parâmetros.

Exemplo de Sobrecarga de Métodos

```
class Calculadora {  
    int soma(int a, int b) {  
        return a + b;  
    }  
  
    double soma(double a, double b) {  
        return a + b;  
    }  
}
```

Vantagens da Sobrecarga

- Mesma operação com diferentes tipos de dados.
- Código mais **organizado e intuitivo**.

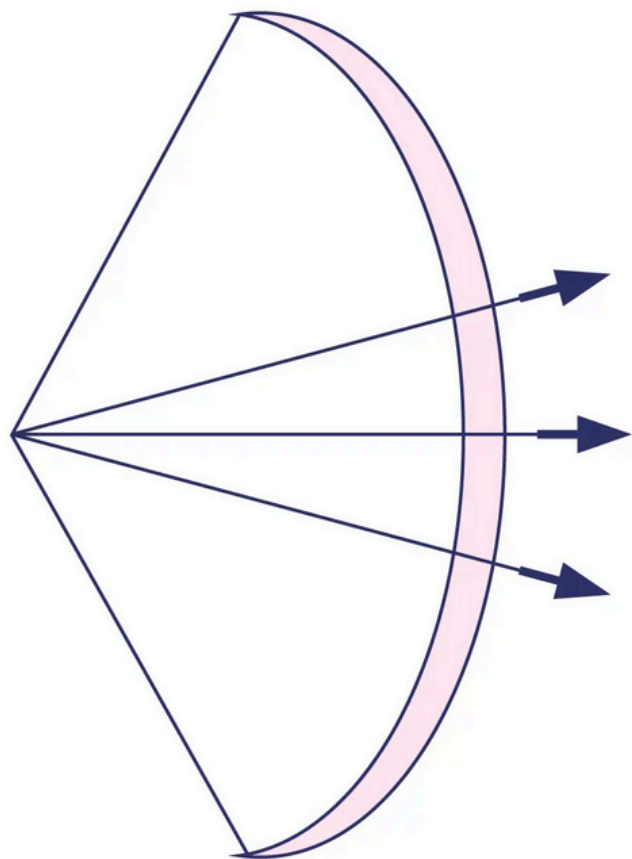
Exemplo Prático

```
Calculadora c = new Calculadora();  
System.out.println(c.soma(3, 4));           // int  
System.out.println(c.soma(2.5, 4.8));       // double
```

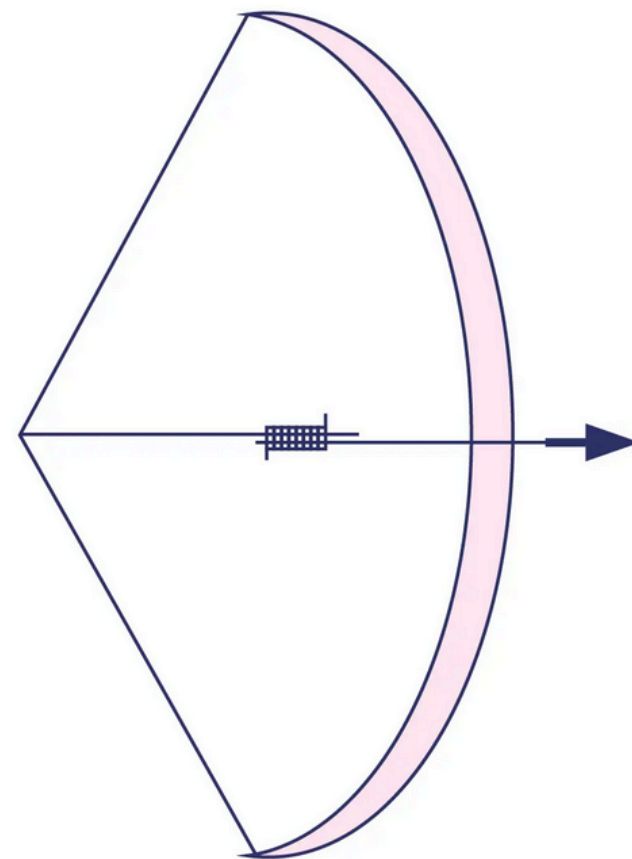
DIFERENÇA ENTRE SOBRECARGA E SOBRESCRITA

- **Sobrecarga (Overloading):** Mesmo nome, parâmetros diferentes, na mesma classe.
- **Sobrescrita (Overriding):** Método redefinido em uma classe filha.

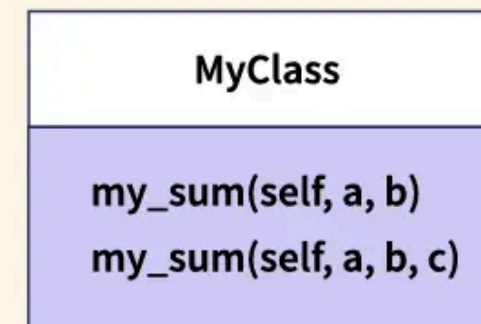
Overloading



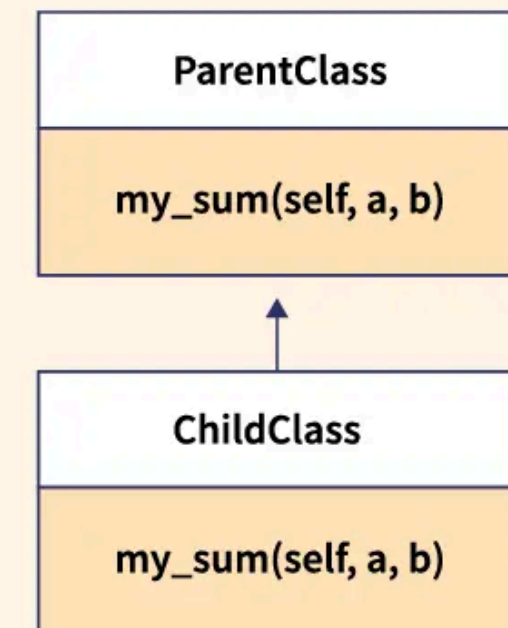
Overriding



Overloading



Overriding



MÉTODOS GET E SET

ATRIBUTOS X MÉTODOS GET E SET

ATRIBUTOS

```
private String nome  
private int idade
```

MÉTODOS GET E SET

```
public String getNome()
```

```
public void  
setNome(String nome)
```

```
public int getIdade()
```

```
public void  
setIdade(int idade)
```

GETTER (GET)

- Serve para **obter/retornar** o valor de um atributo privado.
- Não altera nada, só devolve a informação.
- Exemplo:

```
public int getIdade() {  
    return idade;  
}
```

```
System.out.println(pessoa.getIdade());
```

Mostra a idade da pessoa.

SETTER (SET)

- Serve para **definir ou alterar o valor** de um atributo privado.
- Recebe um parâmetro e atualiza a variável da classe.
- Exemplo:

```
public void setIdade(int idade) {  
    this.idade = idade;  
}
```

```
pessoa.setIdade(25);
```

Altera a idade da pessoa para 25.

GET X SET

- **get** → lê o valor (acesso/leitura).
- **set** → muda o valor (modificação/escrita).

GET

```
public int  
getIdade()
```

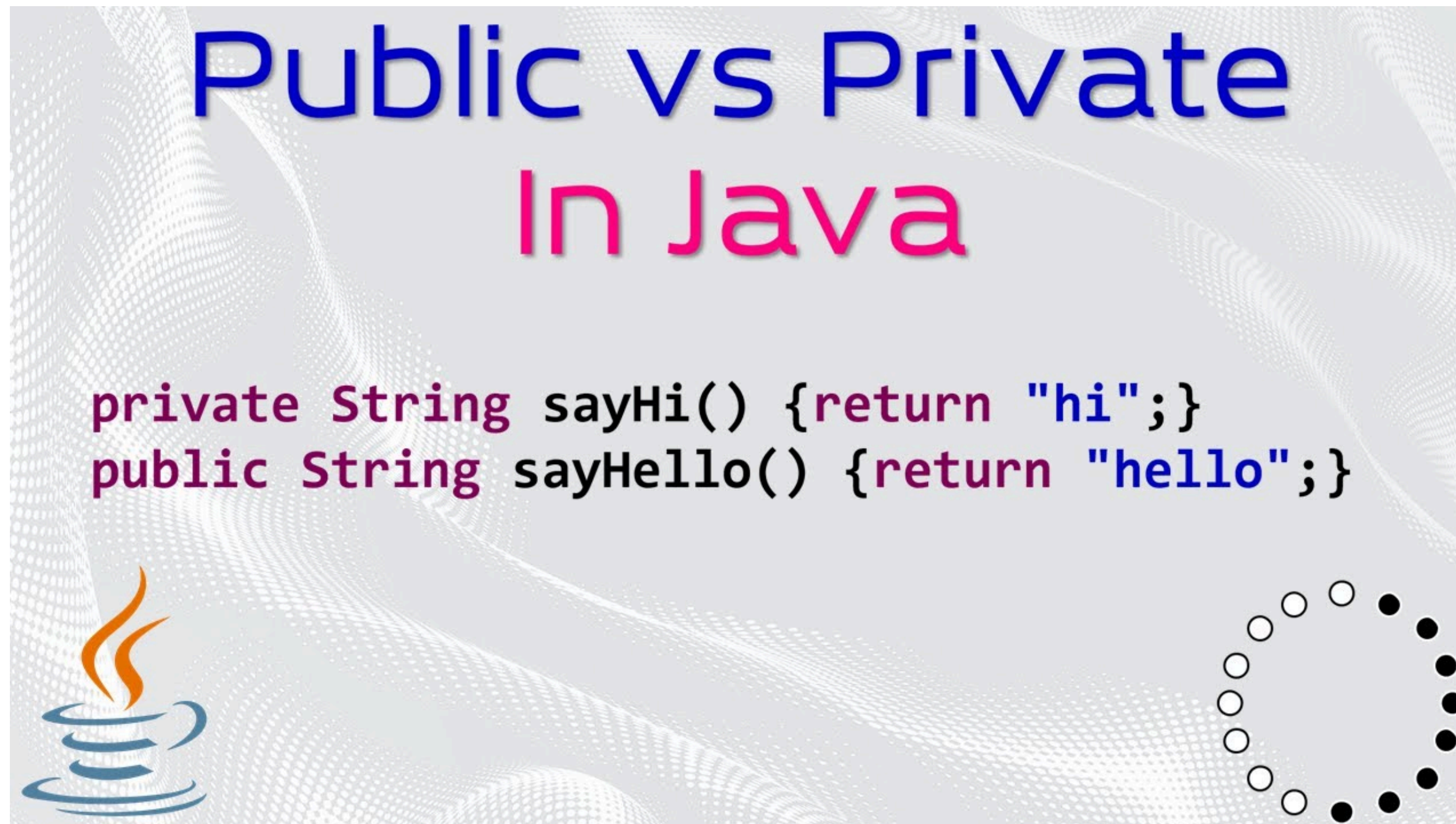
SET

```
public void  
setIdade(int  
idade)
```

Public e Private

PALAVRAS-CHAVE PUBLIC E PRIVATE

Em Programação Orientada a Objetos (POO), **public** e **private** são modificadores de acesso que controlam a visibilidade de atributos, métodos e classes.



PALAVRAS-CHAVE PUBLIC E PRIVATE

PRIVATE

- O acesso é **restrito apenas dentro da própria classe**.
- Ninguém fora da classe consegue acessar diretamente.
- Usado para **proteger os dados** (encapsulamento).
- Normalmente aplicado em **atributos**.

Exemplo:

```
public class Pessoa {  
    private String nome;    // só pode ser acessado dentro da classe  
    private int idade;  
  
    // Métodos públicos para acessar atributos privados  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```

Aqui, **nome** e **idade** estão protegidos. Para acessar, é obrigatório usar **get** e **set**.

PALAVRAS-CHAVE PUBLIC E PRIVATE

PUBLIC

- O acesso é **público**, ou seja, pode ser usado em **qualquer lugar do programa**.
- Normalmente aplicado em **métodos** ou **classes principais**.
- Usado quando queremos que algo seja **visível para todo o código**.

Exemplo:

```
public class Main {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa();  
        p.setNome("Ana");           // acessível  
        System.out.println(p.getNome()); // acessível  
    }  
}
```

Aqui, **setNome()** e **getNome()** são públicos, logo podem ser chamados fora da classe.

PALAVRAS-CHAVE PUBLIC E PRIVATE

Modificador	Onde pode ser usado?	Quem pode acessar?	Uso comum:
private	Atributos e métodos	Somente a própria classe	Proteção de dados (encapsulamento)
public	Classes, métodos e atributos	Qualquer parte do programa	Métodos de acesso (get/set), classes principais

Instanciação de Objetos

INSTANCIACÃO DE OBJETOS

- Criar um objeto na memória usando

```
Pessoa p = new Pessoa();
```

REFERÊNCIA DE OBJETO

- Variável que aponta para o objeto na memória
- Exemplo:

```
Pessoa p1 = new Pessoa();
```

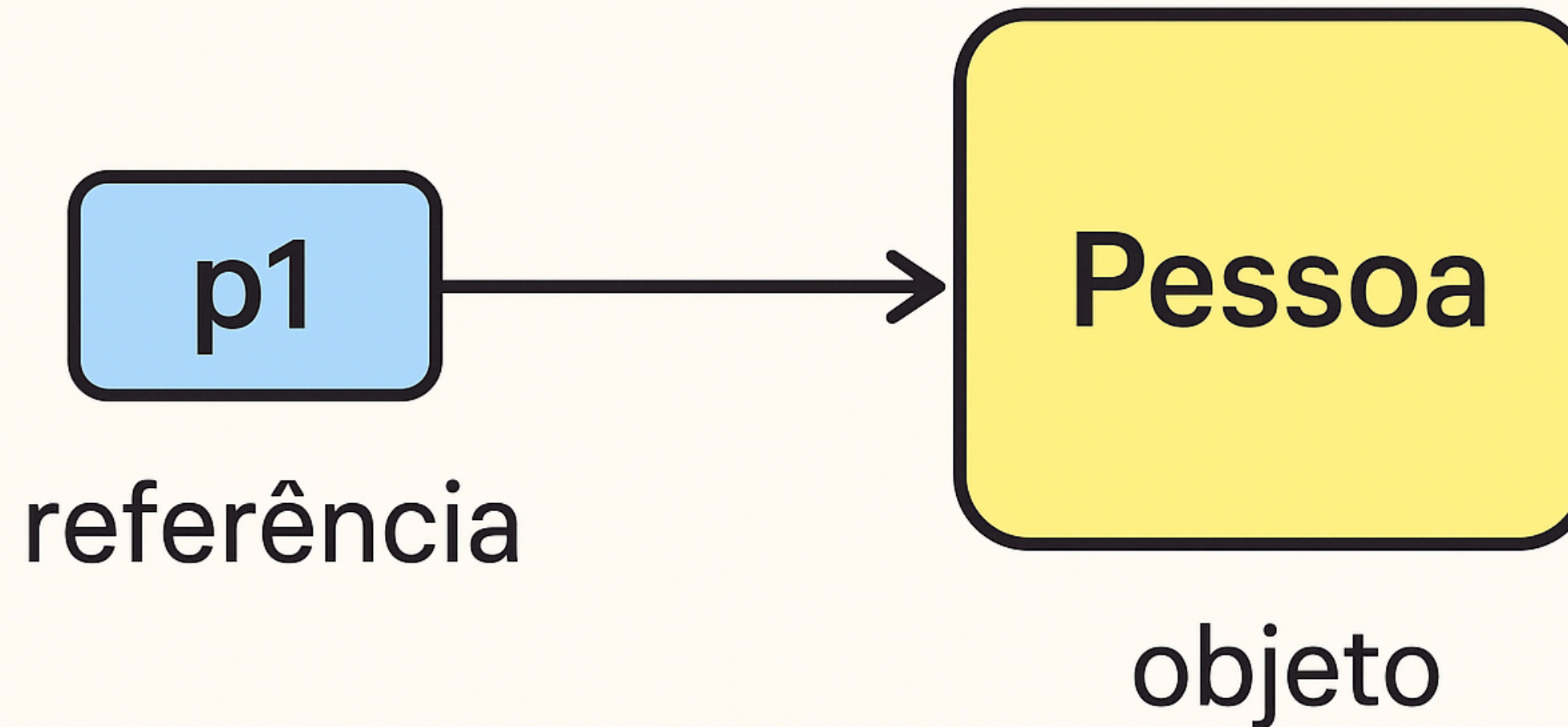
```
Pessoa p2 = p1; // p2 referencia o mesmo objeto que p1
```

DIFERENÇA: OBJETO X REFERÊNCIA

Conceito	Descrição	Exemplo
Objeto	Área de memória com dados	<code>new Pessoa()</code>
Referência	Variável que aponta para objeto	<code>Pessoa p1 = ...</code>

EXEMPLO VISUAL

- Código: **Pessoa p1 = new Pessoa();**
- Exemplo: Caixa **p1** apontando para objeto **Pessoa**



Ciclo de Vida de Um Objeto

CICLO DE VIDA DE UM OBJETO

- Em POO, um objeto em Java passa por algumas etapas principais:

1. **Declaração** – Criamos a referência ao objeto.

- *Carro meuCarro;*

- Aqui, ainda não temos o objeto, só a referência.

2. **Instanciação** – Criamos o objeto em memória com **new**.

- *meuCarro = new Carro();*

3. **Utilização** – Chamamos métodos e acessamos atributos.

- *meuCarro.ligar();*

- *meuCarro.acelerar();*

4. **Finalização (Descarte)** – Quando o objeto não é mais usado, o **Garbage Collector** do Java libera a memória.

Não controlamos isso diretamente, mas podemos usar o método `finalize()` (deprecated, hoje em dia evitado) ou confiar no GC.

EXEMPLO:

```
class Carro {  
    void ligar() {  
        System.out.println("Carro ligado!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Carro c;           // Declaração  
        c = new Carro();    // Instanciação  
        c.ligar();          // Utilização  
        // Após o fim do programa, o objeto é liberado pelo GC  
    }  
}
```

Abstração

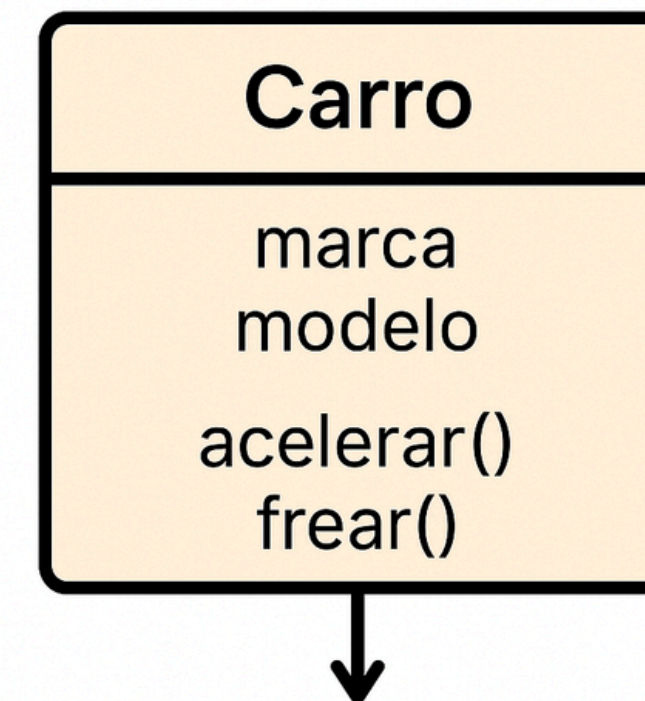
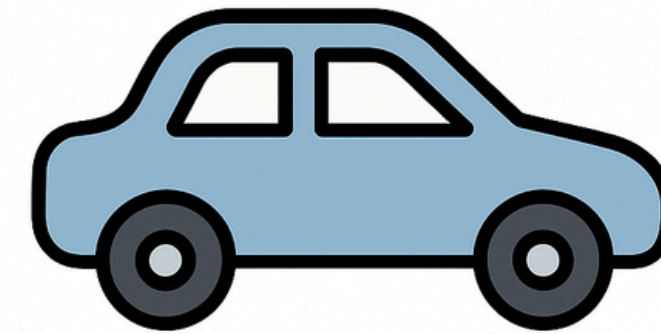
ABSTRAÇÃO

- A abstração é o processo de esconder detalhes de implementação e mostrar apenas o que é relevante.

Benefícios da Abstração:

- Reduz a complexidade, focando apenas no que é relevante
- Facilita a manutenção e evolução do código
- Promove o reuso através de hierarquias de classes
- Permite trabalhar em diferentes níveis de detalhe

Abstração

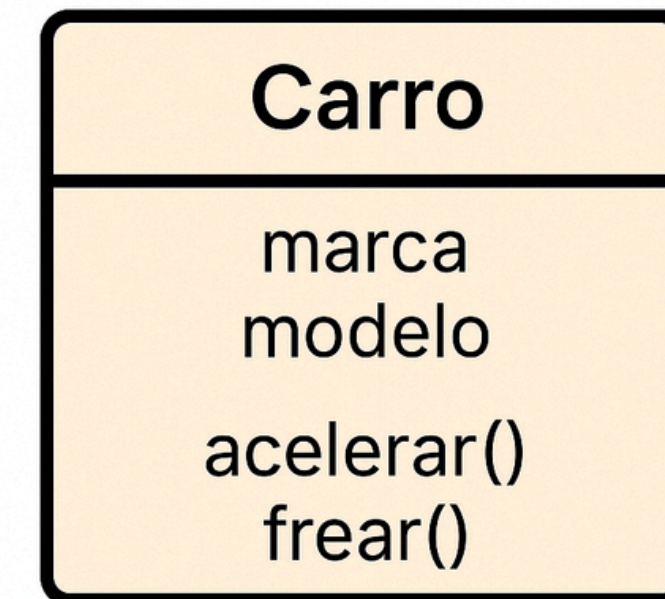
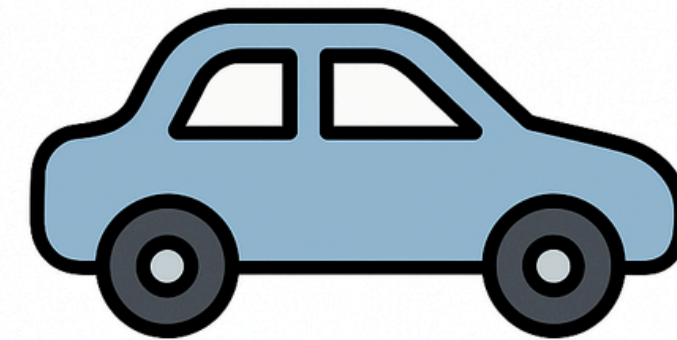


Extrair características
essenciais

ABSTRAÇÃO

- Exemplo: Uma *classe* **Carro** abstrai as características essenciais de um carro real, como **marca**, **modelo** e *métodos* como **acelerar()** e **frear()**, sem se preocupar com detalhes como o funcionamento interno do motor.

Abstração



Extrair características
essenciais

CLASSES ABSTRATAS E INTERFACES

Java oferece dois mecanismos principais para implementar abstração:

1. Uma **classe abstrata** em Programação Orientada a Objetos (POO) é um tipo de classe que não pode ser instanciada diretamente, servindo como um modelo ou base para outras classes (classes derivadas) que a herdam
2. **Interface** é um tipo abstrato usado para declarar um comportamento que as classes devem implementar

CLASSES ABSTRATAS E INTERFACES

Java oferece dois mecanismos principais para implementar abstração:

1) *// Classe abstrata*

```
abstract class Animal {  
    private String nome;  
  
    // Método concreto  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    // Método abstrato (deve ser implementado pelas subclasses)  
    public abstract void emitirSom();  
}
```

2) *// Interface*

```
interface Nadador {  
    void nadar(); // Método implicitamente público e abstrato  
}
```

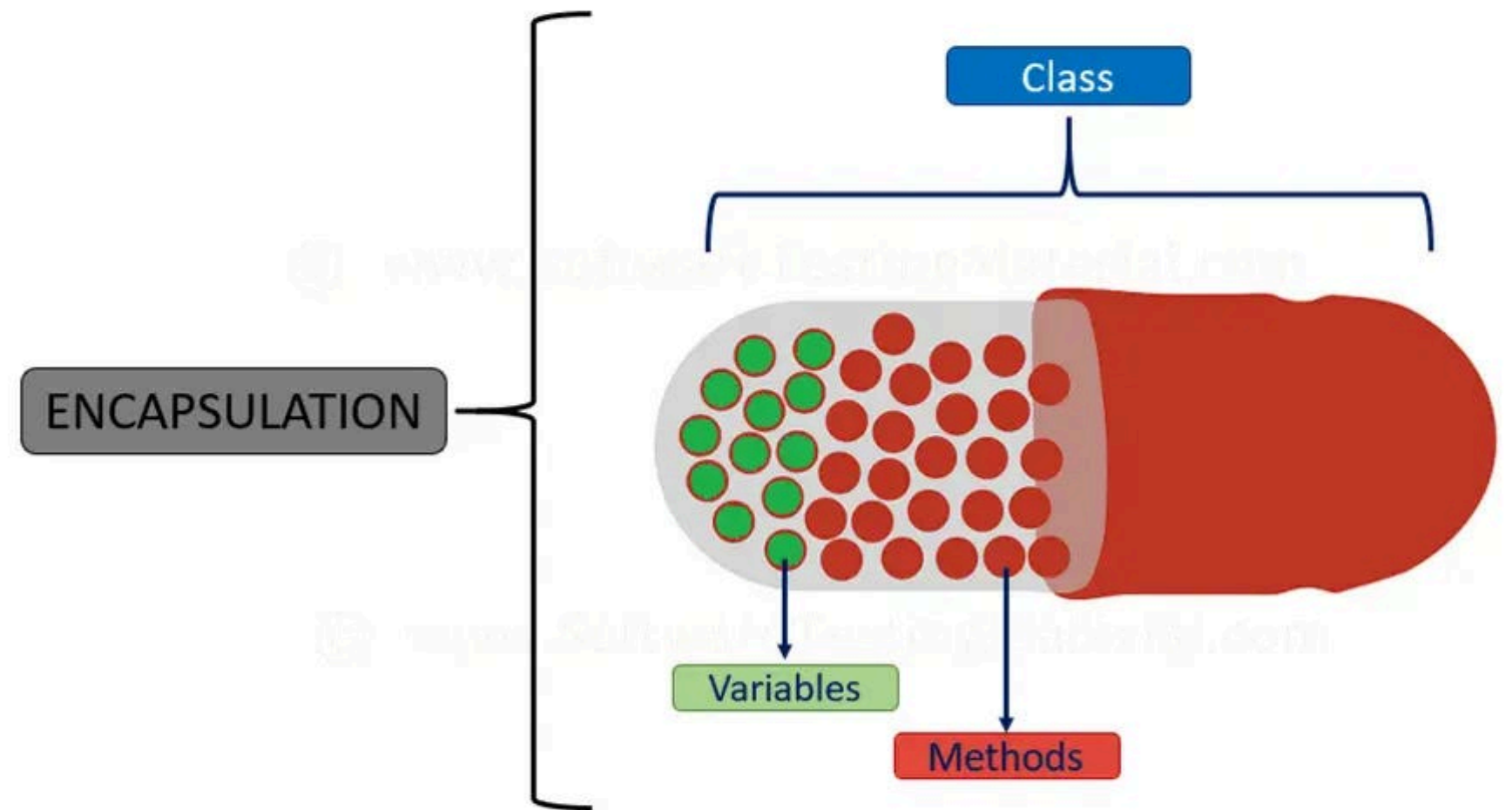
Encapsulamento

ENCAPSULAMENTO

- Encapsulamento é o princípio de ocultar os detalhes de implementação de uma classe e expor apenas o necessário.

Benefícios do Encapsulamento:

- Protege os dados contra acesso não autorizado
- Permite alterar a implementação sem afetar o código cliente
- Garante a validação de dados antes de modificá-los
- Reduz o acoplamento entre componentes do sistema



GETTERS, SETTERS E MODIFICADORES DE ACESSO

- O encapsulamento em Java é implementado principalmente através de:

Modificador	Visibilidade
private	Apenas na própria classe
default	Classes do mesmo pacote
public	Qualquer classe

GETTERS, SETTERS E MODIFICADORES DE ACESSO

- Getters e Setters:

Diferença:

- **setSaldo(double saldo)** → altera diretamente o valor (menos seguro, qualquer número pode ser colocado).
- **depositar(double valor)** → é um **setter especializado**, com validação para impedir valores inválidos.

```
public class ContaBancaria {  
    private double saldo; // Atributo encapsulado  
  
    // Getter  
    public double getSaldo() {  
        return this.saldo;  
    }  
  
    // Setter (simples, sem regras de negócio)  
    public void setSaldo(double saldo) {  
        this.saldo = saldo;  
    }  
}
```

HERANÇA

EXEMPLO:



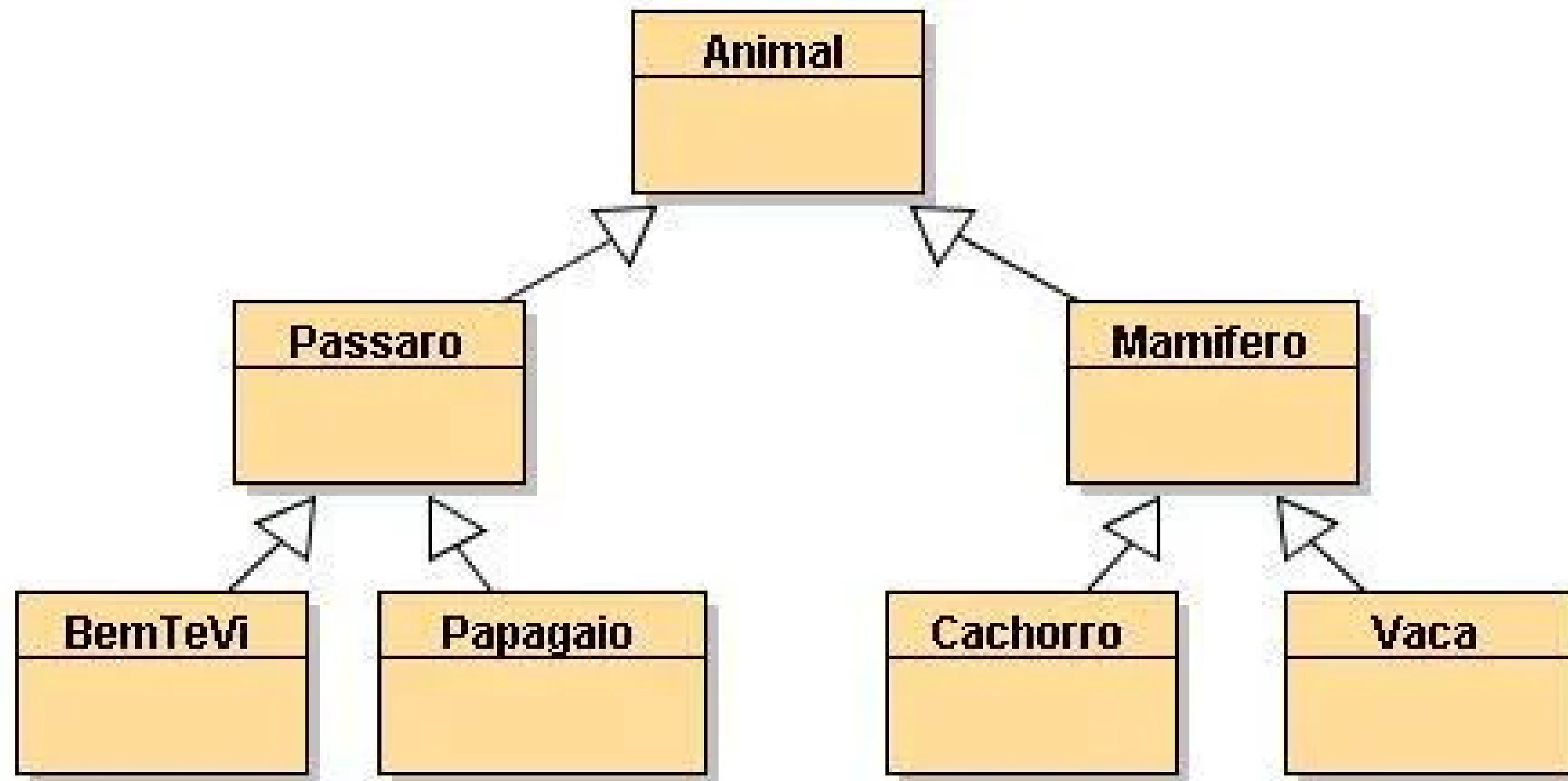
Cachorro herda os **atributos** e **métodos** da **Animal**

```
class Animal {  
    void emitirSom() {  
        System.out.println("Som genérico...");  
    }  
}  
  
class Cachorro extends Animal {  
    void emitirSom() {  
        System.out.println("Latido!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Cachorro();  
        a.emitirSom(); // Saída: Latido!  
    }  
}
```

CRIAÇÃO DE HIERARQUIA DE CLASSES

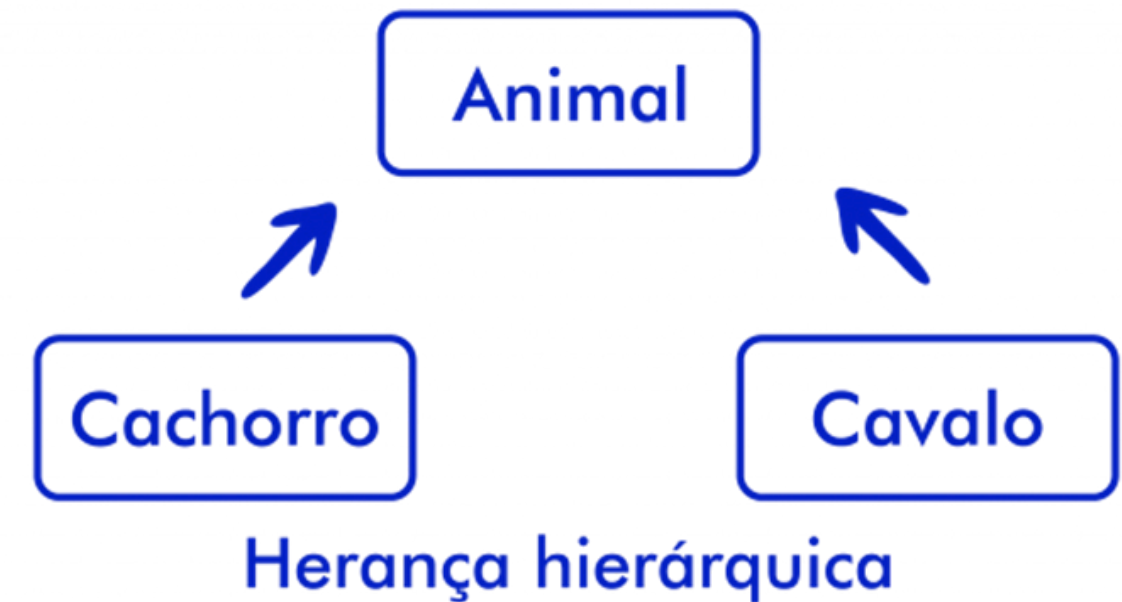
- Uma **hierarquia** organiza classes de forma **genérica** → **específica**.

Exemplo:



O QUE É HERANÇA?

- Herança é um mecanismo da POO que permite criar novas classes a partir de classes existentes.
- A nova classe (subclasse) herda atributos e métodos da classe base (superclasse).
- Promove reutilização de código e organização hierárquica.



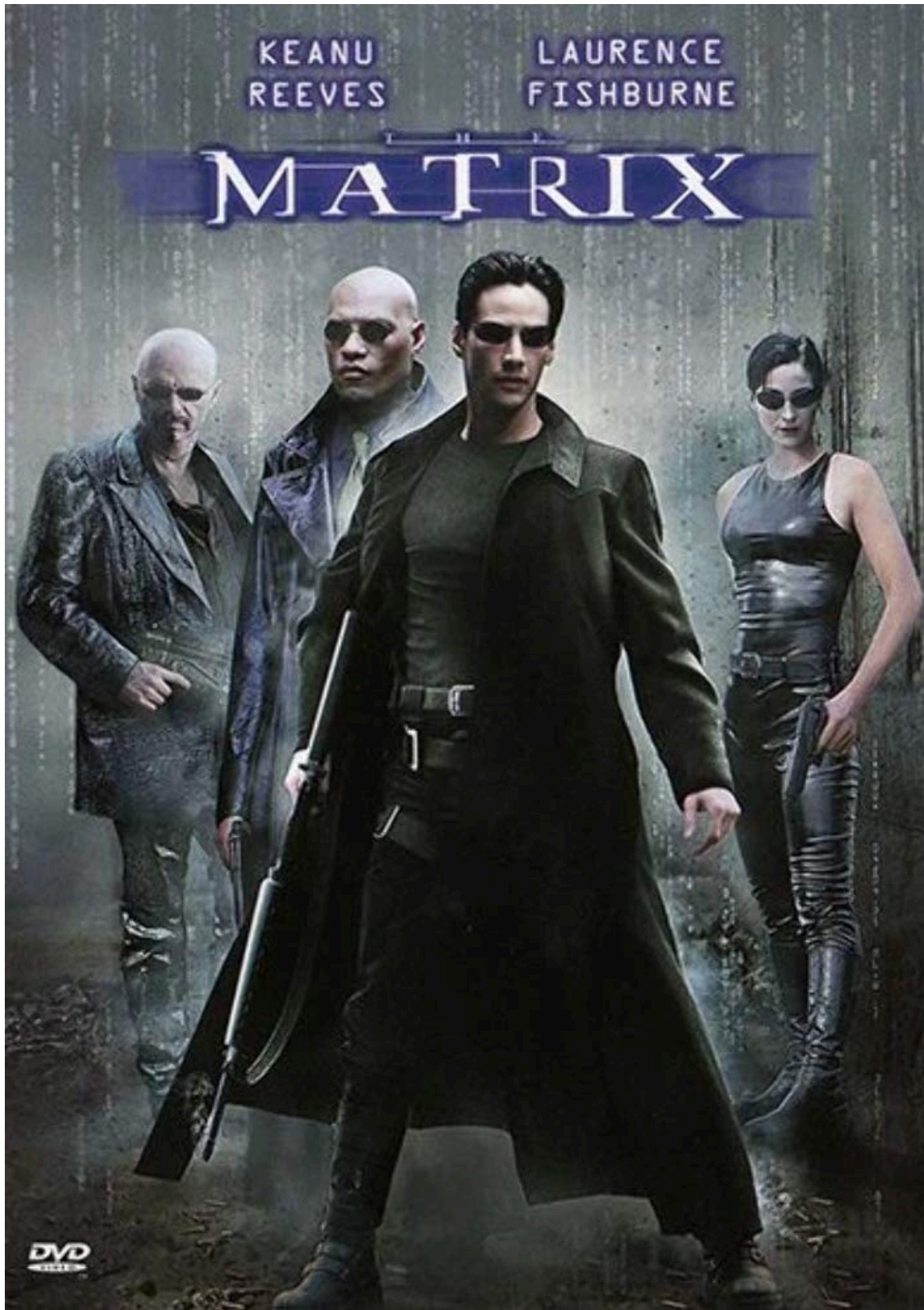
SINTAXE EM JAVA

```
class Superclasse {  
    // atributos e métodos  
}  
  
class Subclasse extends Superclasse {  
    // novos atributos e métodos  
}
```

REFERÊNCIAS

1. BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4. ed. São Paulo: Pearson Prentice Hall, 2008.
2. SANTOS, Rafael. Introdução à Programação Orientada a Objetos usando Java. 2. ed. Rio de Janeiro: Elsevier, 2013. 336 p.
3. SERSON, Roberto Rubinstein. Programação orientada a objetos com Java 6 – curso universitário. 1. ed. Rio de Janeiro: Brasport, jun. 2009. 492 p.
4. PREISS, Bruno R. Estruturas de dados e algoritmos: padrões de projetos orientados a objetos com Java. Rio de Janeiro: Campus, 2001. 566 p.

Dica de Filme: Matrix (1999)



Em um futuro próximo, Thomas Anderson (Keanu Reeves), um jovem programador de computador que mora em um cubículo escuro, é atormentado por estranhos pesadelos nos quais encontra-se conectado por cabos e contra sua vontade, em um imenso sistema de computadores do futuro. Em todas essas ocasiões, acorda gritando no exato momento em que os eletrodos estão para penetrar em seu cérebro. À medida que o sonho se repete, Anderson começa a ter dúvidas sobre a realidade. Por meio do encontro com os misteriosos Morpheus (Laurence Fishburne) e Trinity (Carrie-Anne Moss), Thomas descobre que é, assim como outras pessoas, vítima do Matrix, um sistema inteligente e artificial que manipula a mente das pessoas, criando a ilusão de um mundo real enquanto usa os cérebros e corpos dos indivíduos para produzir energia. Morpheus, entretanto, está convencido de que Thomas é Neo, o aguardado messias capaz de enfrentar o Matrix e conduzir as pessoas de volta à realidade e à liberdade.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code